

UNIT 6: GRAPH REPRESENTATION AND COMPUTATIONAL STRUCTURES

Matrices, Adjacency Structures, Implementation Strategies

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-II



Course Outline

- 1 Graph Matrix Representations
- 2 Matrix Relationships
- 3 Adjacency Structures
- 4 Computational Implementation
- 5 Summary



Outline

- 1 Graph Matrix Representations
- 2 Matrix Relationships
- 3 Adjacency Structures
- 4 Computational Implementation
- 5 Summary



Why Multiple Representations?

Representation Trade-offs

Different graph representations serve *distinct needs*. Each offers unique trade-offs between:

- Memory usage
- Computational speed
- Ease of manipulation

Matrix-Based Representations

Adjacency Matrix

Incidence Matrix

Circuit Matrix

Cut-set Matrix

Path Matrix

adjacency

matrix

1 kb / limit

Social Networks
~ 1×10^6 edges?

operations
↳ steps

Why Multiple Representations?

Representation Trade-offs

Different graph representations serve *distinct needs*. Each offers unique trade-offs between:

- **Memory usage**
- **Computational speed**
- **Ease of manipulation**

Matrix-Based Representations

- Adjacency Matrix
- Incidence Matrix
- Circuit Matrix
- Cut-set Matrix
- Path Matrix

Common
analysis

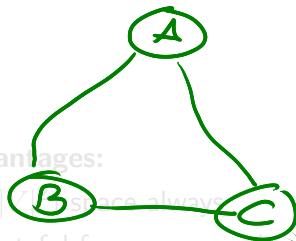
Adjacency Matrix

Definition

A $|V| \times |V|$ boolean matrix where $A[i][j] = 1$ if edge exists between v_i and v_j , else 0.

Example: Undirected Graph with 3 Vertices

	A	B	C
A	0	1	1
B	1	0	1
C	1	1	0



Advantages:

- $O(1)$ edge lookup
- Space operation

Disadvantages:

- $O(|V|^2)$ space always
- Wasteful for sparse graphs

Handwritten red matrix:

	A	B	C	D
A	0	1	1	0
B	1	0	0	0
C	1	0	0	0
D	0	0	0	0



Adjacency Matrix

Definition

A $|V| \times |V|$ **boolean matrix** where $A[i][j] = 1$ if edge exists between v_i and v_j , else 0.

Example: Undirected Graph with 3 Vertices

	A	B	C
A	0	1	1
B	1	0	1
C	1	1	0

Advantages:

- $O(1)$ edge lookup
- Simple operations

Disadvantages:

- $O(|V|^2)$ space always
- Wasteful for sparse graphs

> 1% empty
FO

memory ↑



Incidence Matrix

Definition

A $|V| \times |E|$ matrix where $I[i][j] = 1$ if vertex v_i is incident to edge e_j , else 0.

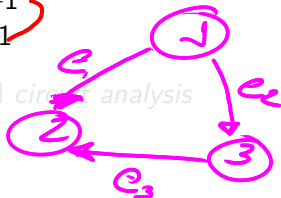
For directed graphs: -1 (tail), $+1$ (head).

Example: Directed Graph

edges (red) *vertices* (red) *2 → 1* (blue) *others → 0* (blue)

	e_1	e_2	e_3
1	-1	-1	0
2	+1	0	+1
3	0	+1	-1

Useful for: Cycle detection and fundamental circuit analysis



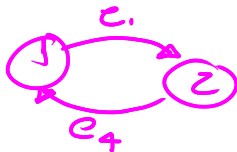
Incidence Matrix

Definition

A $|V| \times |E|$ matrix where $I[i][j] = 1$ if vertex v_i is incident to edge e_j , else 0.

For directed graphs: -1 (tail), $+1$ (head).

Example: Directed Graph



	e_1	e_2	e_3
1	-1	-1	0
2	+1	0	+1
3	0	+1	-1

e_4

+1

cycle

$-1 -1 -1$

Useful for: Cycle detection and fundamental circuit analysis



Circuit and Cut-set Matrices

Circuit Matrix

Derived from fundamental cycles. Each row represents a cycle, each column an edge.

Cut-set Matrix

Represents fundamental cuts (edge sets that disconnect the graph when removed). Derived from spanning tree structure.

Key Property

Circuits and Cut-sets exhibit algebraic duality over finite fields. This relationship underlies matroid theory and network reliability analysis.

	e_1	e_2	e_3	e_4	e_5	e_6	e_7	e_8
C_1	0	1	0	1	0	0	0	0
C_2	1	0	1	0	1	1	0	0
C_3	0	0	0	0	0	0	1	1



Circuit and Cut-set Matrices

Circuit Matrix

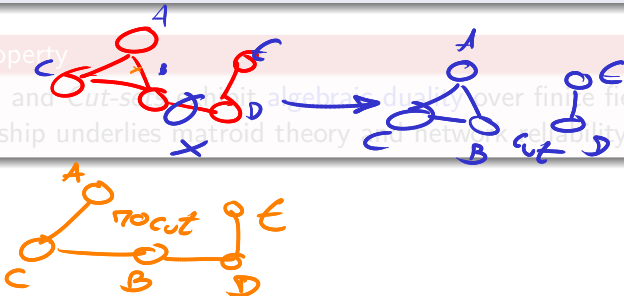
Derived from **fundamental cycles**. Each row represents a cycle, each column an edge.

Cut-set Matrix

Represents **fundamental cuts** (edge sets that disconnect the graph when removed). Derived from spanning tree structure.

Key Property

Circuits and Cut-sets exhibit algebraic duality over finite fields. This relationship underlies matroid theory and network reliability analysis.



Circuit and Cut-set Matrices

Circuit Matrix

Derived from **fundamental cycles**. Each row represents a cycle, each column an edge.

Cut-set Matrix

Represents **fundamental cuts** (edge sets that disconnect the graph when removed). Derived from spanning tree structure.

Key Property

Circuits and Cut-sets exhibit **algebraic duality** over **finite fields**. This relationship underlies **matroid theory** and **network reliability analysis**.

$$\begin{array}{rcl}
 A-B & + & D-E \\
 A-C & + & D-E \\
 A-B & & D-E \\
 A-C & & D-E
 \end{array}$$

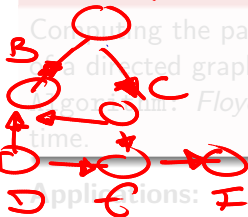


Path Matrix

Definition

A $|V| \times |V|$ matrix where $P[i][j] = 1$ if path exists from v_i to v_j , else 0.

Transitive Closure



	A	B	C	D	E	F
A	1	1	1	1	1	1
B	0	1	1	0	0	0
C	0	1	1	0	1	1
D	0	1	0	1	1	1
E	0	0	0	0	1	1
F	0	0	0	0	0	1

- Strongly connected component detection
- Reachability analysis in DAGs
- Dependency analysis



Path Matrix

Definition

A $|V| \times |V|$ matrix where $P[i][j] = 1$ if path exists from v_i to v_j , else 0.

Transitive Closure

Computing the path matrix is equivalent to finding the transitive closure of a directed graph.

Algorithm: Floyd-Warshall computes all-pairs reachability in $O(|V|^3)$ time.

Applications:

- Strongly connected component detection

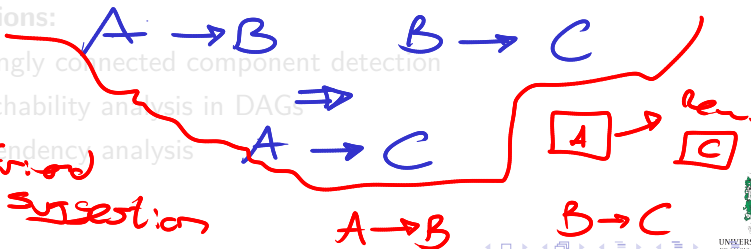
S.N.

Reachability analysis in DAGs

- Dependency analysis

→ Friend

suggestion



Path Matrix

Definition

A $|V| \times |V|$ matrix where $P[i][j] = 1$ if path exists from v_i to v_j , else 0.

Transitive Closure

Computing the path matrix is equivalent to finding the **transitive closure** of a directed graph.

Algorithm: *Floyd-Warshall* computes all-pairs reachability in $O(|V|^3)$ time.

Applications:

- Strongly connected component detection
- Reachability analysis in DAGs
- Dependency analysis

Directed
Acyclic
Graph



Outline

- 1 Graph Matrix Representations
- 2 Matrix Relationships
- 3 Adjacency Structures
- 4 Computational Implementation
- 5 Summary



Converting Between Representations

Interconversions

Matrix representations are *mathematically* related:

- Adjacency → Incidence: Decode edge positions from adjacency structure
- Incidence → Adjacency: Multiply $I \cdot I^T$ (for undirected graphs)
- Adjacency → Path: Compute transitive closure via matrix multiplication

Extracting Structural Information

From matrix representation:

- Degree sequence (row/column sum)
- Connectivity patterns (from *path matrix*)
- Cycle basis (from *spanning tree*)

Converting Between Representations

Interconversions

Matrix representations are *mathematically* related:

- **Adjacency** → **Incidence**: Decode edge positions from adjacency structure
- **Incidence** → **Adjacency**: Multiply $I \cdot I^T$ (for undirected graphs)
- **Adjacency** → **Path**: Compute transitive closure via matrix multiplication

Extracting Structural Information

From any matrix representation:

- Degree sequence (row/column sums)
- Connectivity patterns (from path matrix)
- Cycle basis (from spanning tree)

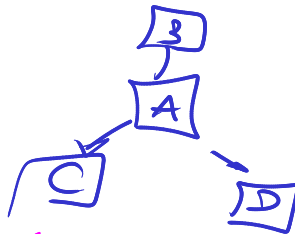
$$\begin{array}{c}
 \begin{array}{ccc}
 0 & 1 & 2 \\
 0 & - & - & - & 1 & - \\
 1 & - & - & - & - & -
 \end{array}
 \end{array}$$

$\Sigma I = 13$



Outline

- 1 Graph Matrix Representations
- 2 Matrix Relationships
- 3 Adjacency Structures
- 4 Computational Implementation
- 5 Summary



keys = vertices

"A" : ["B", "C", "D"]

"B" : [A],

"C" : [A],

"D" : [A]

edges



Adjacency Lists

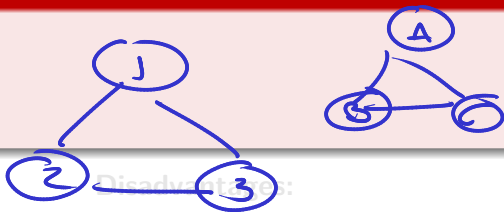
Representation Model

For each vertex, maintain a linked list or array of adjacent vertices.

Space: $O(|V| + |E|)$

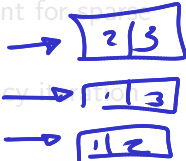
Structure

- Vertex 1: [2, 3]
- Vertex 2: [1, 3]
- Vertex 3: [1, 2]



Advantages:

dict



Disadvantages:

• $O(\deg(v))$ edge lookup

Python

Adjacency Lists

Representation Model

For each vertex, maintain a **linked list** or **array** of adjacent vertices.

Space: $O(|V| + |E|)$

Structure

- Vertex 1: [2, 3]
- Vertex 2: [1, 3]
- Vertex 3: [1, 2]

Advantages:

- Space-efficient for sparse graphs
- Fast adjacency iteration

Disadvantages:

- $O(\deg(v))$ edge lookup
- Less cache-friendly



Adjacency Arrays

Compact Representation

Flatten all adjacency lists into a single contiguous array with index pointers.

Combines spatial locality with list flexibility.

list → linked list → pointers

Structure

Pointers: [0, 2, 4, 6]

Array: [2, 3, 1, 3, 1, 2]

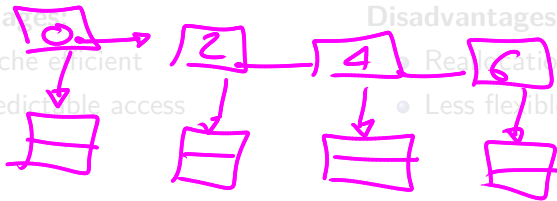
MAX-VERTICES = 10

Advantages:

- Cache efficient
- Predictable access

Disadvantages:

- Reallocation on modification
- Less flexible



Adjacency Arrays

Compact Representation

Flatten all adjacency lists into a single contiguous array with **index pointers**.

Combines spatial locality with list flexibility.

Structure

Pointers: [0, 2, 4, 6]

Array: [2, 3, 1, 3, 1, 2]

Advantages:

- Cache efficient
- Predictable access

Disadvantages:

- Reallocation on modification
- Less flexible

matrix



Outline

- 1 Graph Matrix Representations
- 2 Matrix Relationships
- 3 Adjacency Structures
- 4 Computational Implementation**
- 5 Summary



Implementation Considerations

Language-Specific Choices

- **C/C++:** Manual *memory* + *arrays/pointers* for speed
- **Java/Python:** *HashMap/Dictionary* for adjacency with automatic memory
- **Real-time:** *Adjacency arrays* for predictable cache behavior

hard to code

JS → JSON

list

Pseudocode: Basic Graph Class

key: value

Graph
adjacency: List[]
+ addEdge(u, v)
+ getNeighbors(v)
+ edgeExists(u, v)



Implementation Considerations

Language-Specific Choices

- **C/C++:** Manual *memory + arrays/pointers* for speed
- **Java/Python:** *HashMap/Dictionary* for adjacency with automatic memory
- **Real-time:** *Adjacency arrays* for predictable cache behavior

Pseudocode: Basic Graph Class

Abstract
Data
Type

Graph
adjacency: List[]
+ addEdge(u, v)
+ getNeighbors(v)
+ edgeExists(u, v)

equiv. to
struct C



Memory and Performance Comparison

Space Complexity Comparison

Representation	Space	Edge Lookup
Adjacency Matrix	$O(V ^2)$	$O(1)$
Incidence Matrix	$O(V \cdot E)$	$O(V)$
Adjacency List	$O(V + E)$	$O(\deg(v))$
Adjacency Array	$O(V + E)$	$O(\log \deg(v))$

Selection Guide

- **Dense graphs** ($|E| \approx |V|^2$): Use adjacency matrix
- **Sparse graphs** ($|E| \ll |V|^2$): Use adjacency lists
- **General purpose**: Adjacency lists are best balance



Memory and Performance Comparison

Space Complexity Comparison

Representation	Space	Edge Lookup
Adjacency Matrix	$O(V ^2)$	$O(1)$
Incidence Matrix	$O(V \cdot E)$	$O(V)$
Adjacency List	$O(V + E)$	$O(\deg(v))$
Adjacency Array	$O(V + E)$	$O(\log \deg(v))$

Selection Guide

- **Dense graphs** ($|E| \approx |V|^2$): Use adjacency matrix
- **Sparse graphs** ($|E| \ll |V|^2$): Use adjacency lists
- **General purpose**: Adjacency lists are best balance



Algorithm Performance

Operation Cost Analysis

Operation	Matrix	List	Array
Edge exists?	$O(1)$	$O(\deg v)$	$O(\log \deg v)$
Add edge	$O(1)$	$O(1)$	$O(1)$ amortized
Remove edge	$O(1)$	$O(\deg v)$	$O(\deg v)$
List neighbors	$O(V)$	$O(1)$	$O(1)$
BFS/DFS	$O(V ^2)$	$O(V + E)$	$O(V + E)$

Recommendation

Adjacency lists provide the best balance for most real-world applications.



Outline

- 1 Graph Matrix Representations
- 2 Matrix Relationships
- 3 Adjacency Structures
- 4 Computational Implementation
- 5 Summary



Key Takeaways

Graph Matrices

Provide mathematical richness for theoretical analysis but *consume quadratic space*.

Adjacency Structures

Lists and arrays offer *linear space complexity* and efficient algorithm support.

Selection Criteria

Choose based on:

- Graph density
- Operation frequency
- Memory constraints
- Cache behavior requirements

Key Takeaways

Graph Matrices

Provide **mathematical richness** for theoretical analysis but *consume quadratic space*.

Adjacency Structures

Lists and arrays offer linear space complexity and efficient algorithm support.

Selection Criteria

Choose based on:

- Graph density
- Operation frequency
- Memory constraints
- Cache behavior requirements

Key Takeaways

Graph Matrices

Provide **mathematical richness** for theoretical analysis but *consume quadratic space*.

Adjacency Structures

Lists and arrays offer *linear space complexity* and efficient algorithm support.

Selection Criteria

Choose based on:

- Graph density
- Operation frequency
- Memory constraints
- Cache behavior requirements

Outline

- 1 Graph Matrix Representations
- 2 Matrix Relationships
- 3 Adjacency Structures
- 4 Computational Implementation
- 5 Summary



Thanks!

Questions?

